

HW-2

Ans. to the Ques. no-01

Lexemes ⇒

Lexemes are the lowest-level syntactic units or the actual character sequence in a source program that match a token pattern.

Example ⇒ `int sum = a + 25 ;`

'int', 'sum', '=', 'a', '25', ';' are lexemes.

Token ⇒

Token is a category or classification assigned to a lexeme by the lexical analyzer.

Example ⇒ `int sum = a + 25 ;`

'int' ⇒ KEYWORD

'sum' ⇒ IDENTIFIER

'=' ⇒ ASSIGNMENT_OP

'a' ⇒ IDENTIFIER

'+' ⇒ ADD_OP

'25' ⇒ INTEGER

';' ⇒ SEMICOLON

Reserved Words ⇒

Reserved words in programming language have predefined meaning and that's why can't be assigned as identifiers or variable names.

Example ⇒ `if, switch, while`

Metalanguage $\Rightarrow$

Metalanguage is used to define other languages, the syntax of other languages.

Example $\Rightarrow$ BNF (Backus Naur Form), EBNF are metalanguages.

$\langle \text{digit} \rangle ::= 0 | 1 | 2 | 3 | \dots | 9$

Context Free Grammar $\Rightarrow$

context-free grammar is a formal system used to describe the syntax of a language.

There might be - terminals, non-terminals, production rules etc.

Example $\Rightarrow$

$\langle s \rangle = \langle s \rangle b$

$\langle s \rangle = a | \epsilon$

$S \rightarrow Sb | a | \epsilon$

Derivation $\Rightarrow$

Derivation is the sequence of production steps to generate a target symbol from the start symbol repeatedly by production rules.

Example $\Rightarrow$ If $S \rightarrow Sb | a | \epsilon$
To derive $abbb$
 $S \rightarrow Sb$
 $\Rightarrow Sb$
 $\Rightarrow Sbb$
 $\Rightarrow Sbbb$
 $\Rightarrow abbb$

Production Rule $\Rightarrow$

Production rule specifies how a non-terminal can be changed by terminals or non-terminals.

Example $\Rightarrow \langle \text{expr} \rangle = \langle \text{expr} \rangle + \langle \text{term} \rangle | \langle \text{expr} \rangle - \langle \text{term} \rangle$

Ans. to the ques- no-02

BNF (Backus Naur Form) is a notation used to describe the syntax of programming language and EBNF (Enhanced Backus Naur Form) provides additional notation that makes grammars shorter, cleaner, and easier to understand.

There are certain advantages of EBNF over BNF $\Rightarrow$

- Higher Readability $\Rightarrow$ EBNF eliminates excessive recursion by introducing repetition brackets, making rules shorter and simpler.
- Conciseness $\Rightarrow$ EBNF reduces the total number of production rules through optional elements and groupings.
- Standard Operators $\Rightarrow$ EBNF brings REGEX like extensions directly into grammar definitions.
Example - $\{ \}$ or $*$ for repetition (0 or more times)

For the conversion of BNF to EBNF

The example would be, $\langle \text{list} \rangle ::= \langle \text{item} \rangle | \langle \text{item} \rangle, \langle \text{list} \rangle$
 $\langle \text{item} \rangle ::= a | b | c$

So, the list can contain multiple items separated by commas.

Using recursive repetition, EBNF would be,

$\langle \text{list} \rangle ::= \langle \text{Item} \rangle \{ ", " \langle \text{item} \rangle \}$

$\langle \text{item} \rangle ::= "a" \mid "b" \mid "c"$

Here, a list can contain 1 or more items.

If there are more items, they would

be separated by commas. And, items

can either be A, B or c.

Ans. to the Ques. no. 03

Given that,

$\langle S \rangle \rightarrow \langle A \rangle a \langle B \rangle b$

$\langle A \rangle \rightarrow \langle A \rangle b \mid b$

$\langle B \rangle \rightarrow b$

Using left-most derivation,

The first string $s = b a b b$,

$S = A a B b$

$= b a B b \quad ; [A = b]$

$= b a b b \quad ; [B = b]$

So, the first string is derivable.

The second string, $S = b b b a b b$,

$S = A a B b$

$= A b a B b \quad ; [A = A b]$

$= A b b a B b \quad ; [A = A b]$

$= b b b a B b \quad ; [A = b]$

$= b b b a b b \quad ; [B = b]$

So, the second string is derivable as well,

For the third string, $s = bb\ aaa\ aabc$,

Here, ~~as~~ per the definition of NS , there can't be any 'c' in the string in any way.

For the fourth string, $S = aaaaaa$,

this is also not derivable as the string accepted (minimum) might be babb. There is no definition of the grammar which allows all 'a's'.

Ans. to the Ques. no-04

The language - includes all strings containing n copies of alphabet a , followed by $n+1$ copies of the alphabet b , assuming $n > 0$.

$S \rightarrow \{a^n b^{n+1} \mid n \geq 0\}$
 $\rightarrow a^n b^{n+1}$ as $b^{n+1} = b^n b$
 $\rightarrow a^n b^n b$ [as $n \geq 0$]
 $\rightarrow a^n b^n$ base case

Here,

$$\langle s \rangle \rightarrow a \langle s \rangle b \mid abb$$

Here, abb is the base case for the language, as $n > 0$. And as, $\langle S \rangle$ already contains $a^n b^{n+1}$, we can just extend it by concatenating

an 'a' in the front and 'b' in the back.

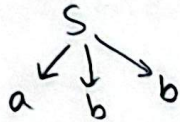
Using left-most derivation,

For the first sentence - $abbb$,

$$S = abbb$$

$\Rightarrow$ This is the base case.

The parse tree would be,



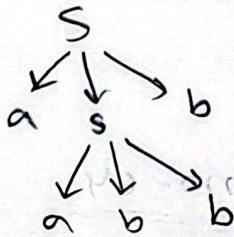
For the second sentence - $aabbbb$,

$$S = aSb$$

$$= aabbbb$$

; $[S = abbb]$

The parse tree would be,



Ans. to the ques. no-05

For the hypothetical programming language,

an identifier would be strings of

consist of letters and numbers, and

they must begin with a letter.

$\langle \text{identifier} \rangle = \langle \text{letter} \rangle \langle \text{number} \rangle$

$\langle \text{number} \rangle = \langle \text{letter} \rangle \langle \text{digit} \rangle \mid \langle \text{digit} \rangle \langle \text{number} \rangle \mid \epsilon$

$\langle \text{letter} \rangle = a \mid b \mid \dots \mid z$

$\langle \text{digit} \rangle = 0 \mid 1 \mid \dots \mid 9$

$\epsilon = \text{empty string}$

This is the EBNF version -

$\langle \text{identifier} \rangle ::= \langle \text{letter} \rangle \{ \langle \text{letter} \rangle \mid \langle \text{digit} \rangle \}^*$

So, it always starts with a letter.

Ans. to the Ques. No-06

A grammar is ambiguous when we can build more than one parse tree for a particular string given - which makes the compiler confused.

An example might be:

$S \rightarrow AB \mid C$

$A \rightarrow aAB \mid ab$

$B \rightarrow cBd \mid cd$

$C \rightarrow aCd \mid aDd$

$D \rightarrow bDc \mid bc$

an example string would be, $S = aabbccdd$

Using left most derivation,

$$S \rightarrow AB$$

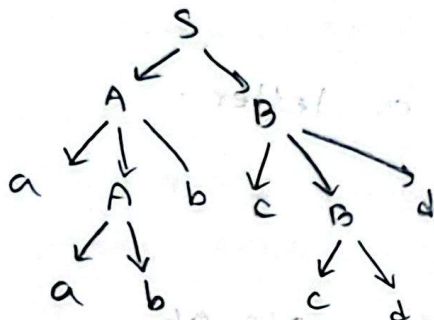
$$\rightarrow aAbB \quad ; [A = aAb]$$

$$\rightarrow aabbB \quad ; [A = ab]$$

$$\rightarrow aabbCBd \quad ; [B = CBd]$$

$$\rightarrow aabbccdd \quad ; [B = cd]$$

So, the first parse tree becomes,



For the second derivation,

$$S \rightarrow C$$

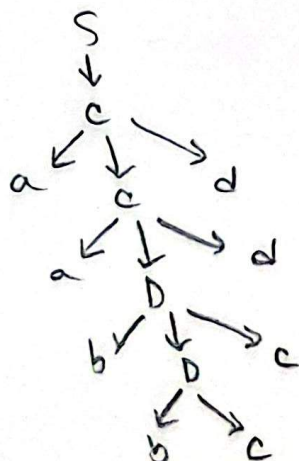
$$\rightarrow aCd \quad ; [C = aCd]$$

$$\rightarrow aaDdd \quad ; [C = aDd]$$

$$\rightarrow aabbDcdd \quad ; [D = bDc]$$

$$\rightarrow aabbccdd \quad ; [D = bc]$$

So, the second parse tree becomes,



For that particular string, we could generate two distinct parse trees, which is a prone that the grammar is ambiguous.

Ans. to the ques. no-07

Three control-flow statements in C,

- (i) if
- (ii) ~~if else~~ for
- (iii) while

(i) For the 'if' statement,

we write in C as,

```
if (condition)
{
    statement;
}
```

The grammar would be,
~~<expression>~~ ~~<expression>~~
~~<statement>~~
 $\langle \text{if-statement} \rangle = \text{if} (\langle \text{expression} \rangle$
 $\langle \text{statement} \rangle$

Here, if and the brackets () are terminals
and $\langle \text{expression} \rangle$, $\langle \text{statement} \rangle$ are non-terminals.

(ii) For the 'for' statement,

we write in C as,

```
for (initialization ; condition ; update)
{
    statement;
}
```

The grammar would be
 $\langle \text{for-statement} \rangle$
 $= \text{for} (\langle \text{init} \rangle ; \langle \text{expr} \rangle ; \langle \text{update} \rangle$
 $\langle \text{statement} \rangle$

Here, for and () are terminals and $\langle \text{init} \rangle$, $\langle \text{expr} \rangle$, $\langle \text{update} \rangle$, $\langle \text{statement} \rangle$ are non-terminals.

③ For the 'while' statement,

we write in C as,

```
while (condition)
{
    statement
}
```

The grammar would be,

$\langle \text{while-statement} \rangle = \text{while}(\langle \text{condition} \rangle)$
 $\langle \text{statement} \rangle$

Here, 'while', '(', ')' are terminals and
 $\langle \text{condition} \rangle$, $\langle \text{statement} \rangle$ are non-terminals.